

Dive into Python Library

실전 파이썬 라이브러리 활용

02 라이브러리 설치하기

목차 02 라이브러리 설치하기

1. 가상환경이란
2. 가상환경의 필요성
3. 가상환경 활용하기
4. 라이브러리 설치 – PyPI
5. 라이브러리 설치 – 파일
6. 2장 정리 및 3장 예고

1. 가상환경이란

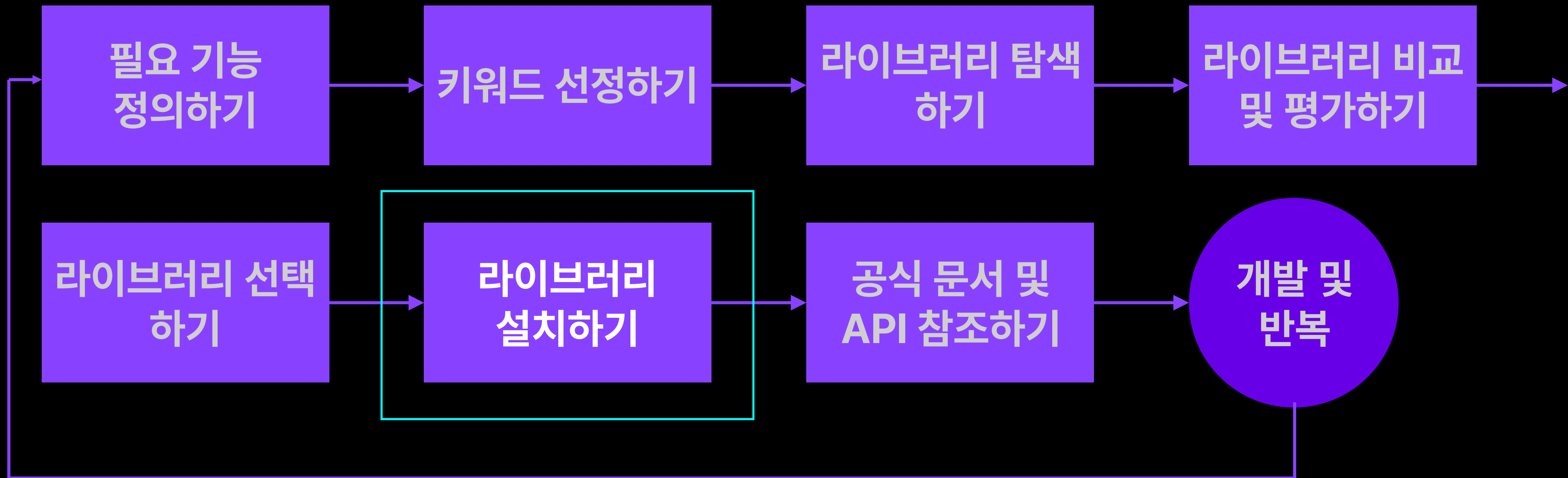


라이브러리 탐색 과정

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

1. 가상환경이란





1. 가상환경

2. PyPI에서 설치

3. 파일로 설치

가상환경(Virtual Environment) - 파이썬

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

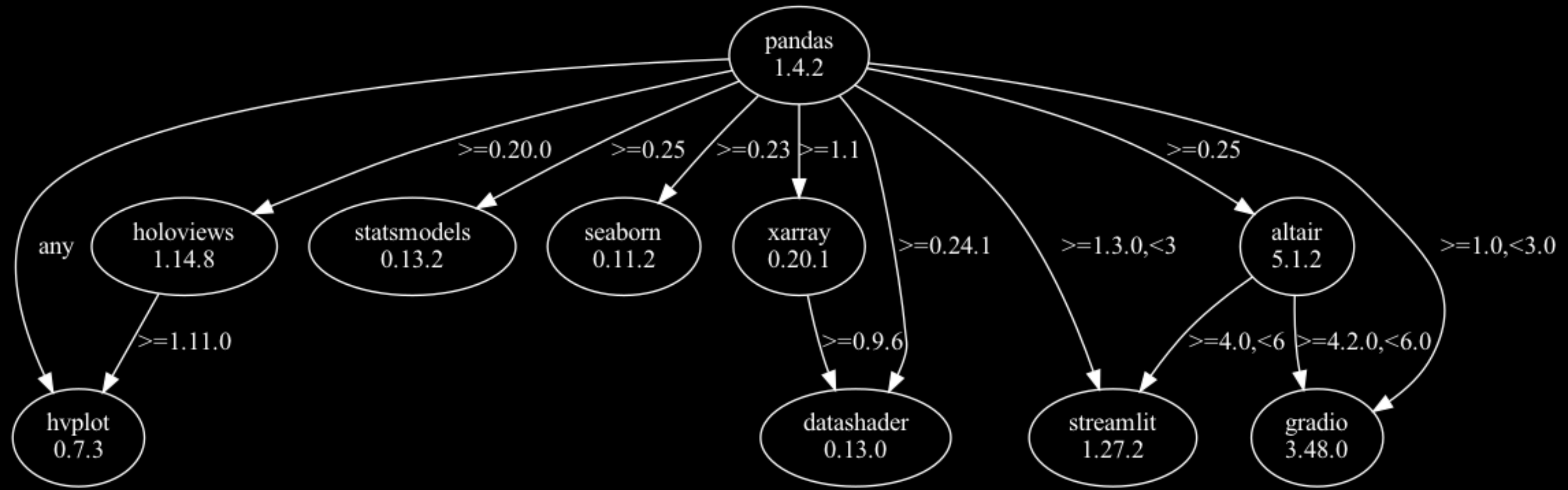
1. 가상환경이란

정의

- 독립적인 파이썬 실행 환경

사용하는 이유

- 각각의 프로젝트가 필요로 하는 라이브러리와 패키지를 격리된 환경에서 관리할 수 있음
- 프로젝트 간의 의존성 충돌을 방지하고 개발 환경을 관리하는 데 도움을 줌



= 한 라이브러리가 다른 라이브러리에 의존하는 관계

파이썬 프로젝트에서 **A 라이브러리**가 **B 라이브러리**에 의존한다면,
A 라이브러리를 사용하려면 **B 라이브러리**도 함께 설치해야 정상적으로 작동

2. 가상환경의 필요성



1. 버전 문제

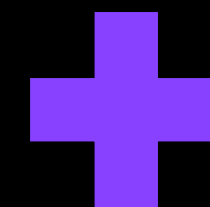
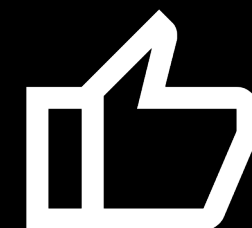
실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

2. 가상환경의 필요성

site-packages

pandas 1.5.3
(2023.01.19)



프로그램 A

프로그램 B

pandas >= 2.2.0



1. 버전 문제

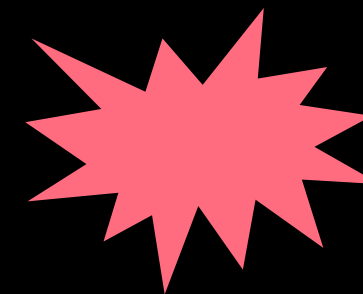
실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

2. 가상환경의 필요성

site-packages

pandas 2.2.0
(2024.01.20)

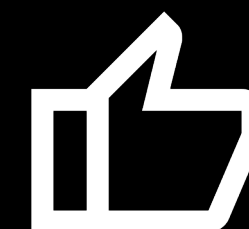


~~프로그램 A~~



프로그램 B

pandas >= 2.2.0





버전 문제의 발생 원인

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

2. 가상환경의 필요성

- 라이브러리들은 지속적으로 유지, 보수됨
- 따라서, 기존 버전의 라이브러리에 있는 기능들이 없어질 수도, 새로 개발될 수 있음
- 이러한 변경 사항들은 모두 공식 홈페이지 혹은 PyPI release note를 통해 사용자들에게 공유

The screenshot shows the pandas library website. The top navigation bar includes links for 'Getting started', 'User Guide', 'API reference', 'Development', and 'Release notes' (which is highlighted). A search bar is also present. The main content area is titled 'Deprecations' and lists several items:

- Deprecated parsing datetime strings with system-local timezone to `tzlocal`, pass a `tz` keyword or explicitly call `tz_localize` instead ([GH 50791](#))
- Deprecated argument `infer_datetime_format` in `to_datetime()` and `read_csv()`, as a strict version of it is now the default ([GH 48621](#))
- Deprecated behavior of `to_datetime()` with `unit` when parsing strings, in a future version these will be parsed as datetimes (matching unit-less behavior) instead of cast to floats. To retain the old behavior, cast strings to numeric types before calling `to_datetime()` ([GH 50735](#))
- Deprecated `pandas.io.sql.execute()` ([GH 50185](#))
- `Index.is_boolean()` has been deprecated. Use `pandas.api.types.is_bool_dtype()` instead ([GH 50042](#))
- `Index.is_integer()` has been deprecated. Use `pandas.api.types.is_integer_dtype()` instead ([GH 50042](#))

On the left side of the page, there is a list of 'What's new' for various versions of pandas, with 'What's new in 2.0.0 (April 3, 2023)' highlighted.



pandas Getting started User Guide API reference Development Release notes Search % + K

Deprecations

- Deprecated parsing datetime strings with system-local timezone to `tzlocal`, pass a `tz` keyword or explicitly call `tz_localize` instead ([GH 50791](#))
- Deprecated argument `infer_datetime_format` in `to_datetime()` and `read_csv()`, as a strict version of it is now the default ([GH 48621](#))
- Deprecated behavior of `to_datetime()` with `unit` when parsing strings, in a future version these will be parsed as datetimes (matching unit-less behavior) instead of cast to floats. To retain the old behavior, cast strings to numeric types before calling `to_datetime()` ([GH 50735](#))
- Deprecated `pandas.io.sql.execute()` ([GH 50185](#))
- `Index.is_boolean()` has been deprecated. Use `pandas.api.types.is_bool_dtype()` instead ([GH 50042](#))
- `Index.is_integer()` has been deprecated. Use `pandas.api.types.is_integer_dtype()` instead ([GH 50042](#))

개발 분야에서 특정 기능, 기술, 라이브러리, API 등이 더 이상 권장되지 않고,
향후 버전에서 제거될 예정임을 나타내기 위해 사용



futureWarning을 발생시킴



Removal of prior version deprecations/changes

- Removed `Int64Index`, `UInt64Index` and `Float64Index`. See also [here](#) for more information ([GH 42717](#))
- Removed deprecated `Timestamp.freq`, `Timestamp.freqstr` and argument `freq` from the `Timestamp` constructor and `Timestamp.fromordinal()` ([GH 14146](#))
- Removed deprecated `CategoricalBlock`, `Block.is_categorical()`, require `datetime64` and `timedelta64` values to be wrapped in `DatetimeArray` or `TimedeltaArray` before passing to `Block.make_block_same_class()`, require `DatetimeTZBlock.values` to have the correct `ndim` when passing to the `BlockManager` constructor, and removed the “fastpath” keyword from the `SingleBlockManager` constructor ([GH 40226](#), [GH 40571](#))
- Removed deprecated global option `use_inf_as_null` in favor of `use_inf_as_na` ([GH 17126](#))
- Removed deprecated module `pandas.core.index` ([GH 30193](#))

실제 삭제되는 시점은 버전 정책,
공지 기간, 대체 기능,
커뮤니티 피드백 등에 따라 다름.

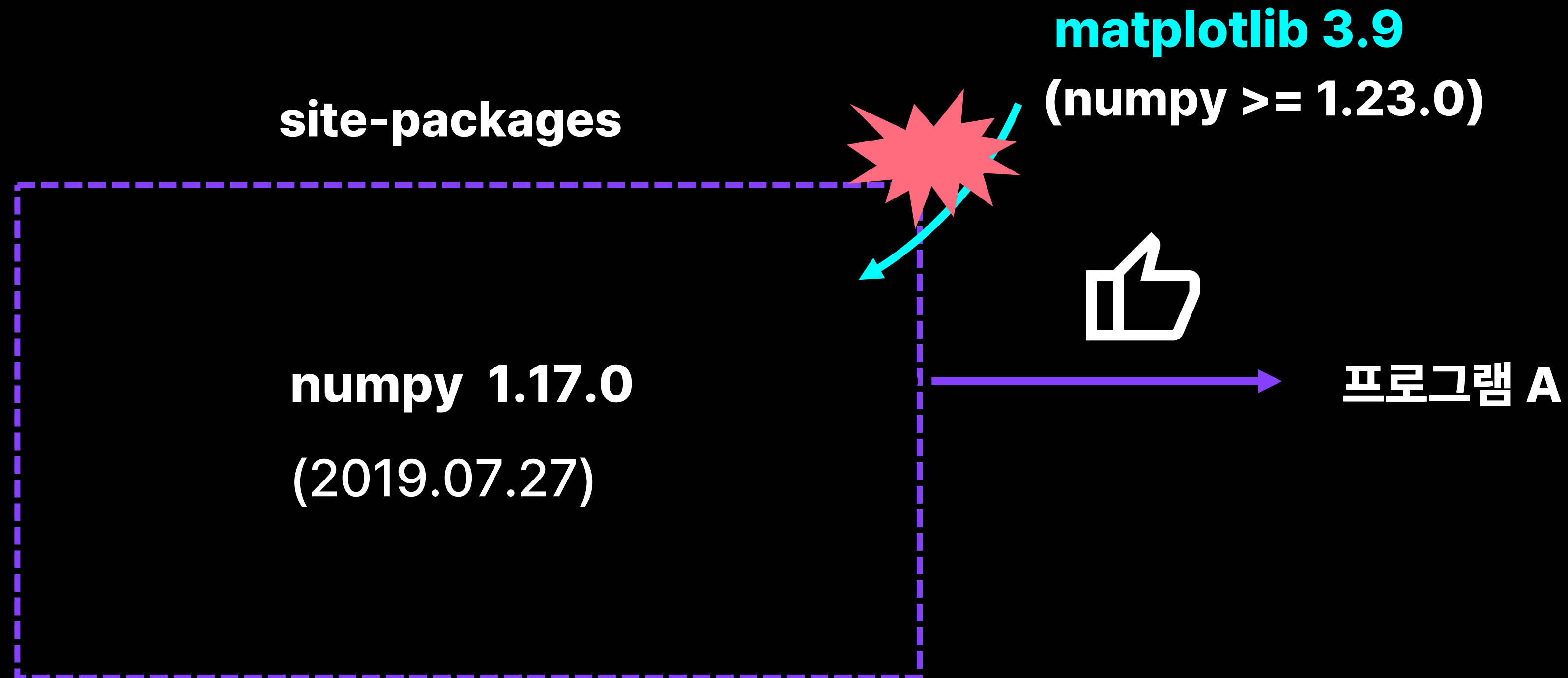


2. 의존성 문제

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

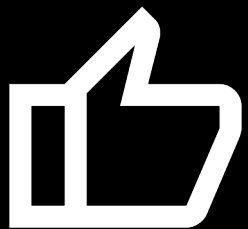
2. 가상환경의 필요성





3. 무거운 환경

site-packages

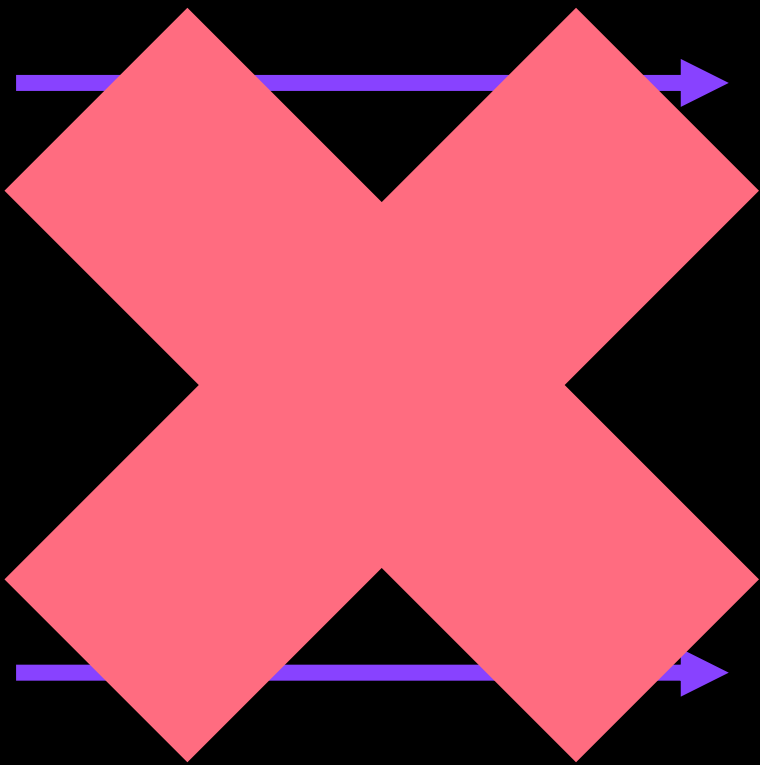
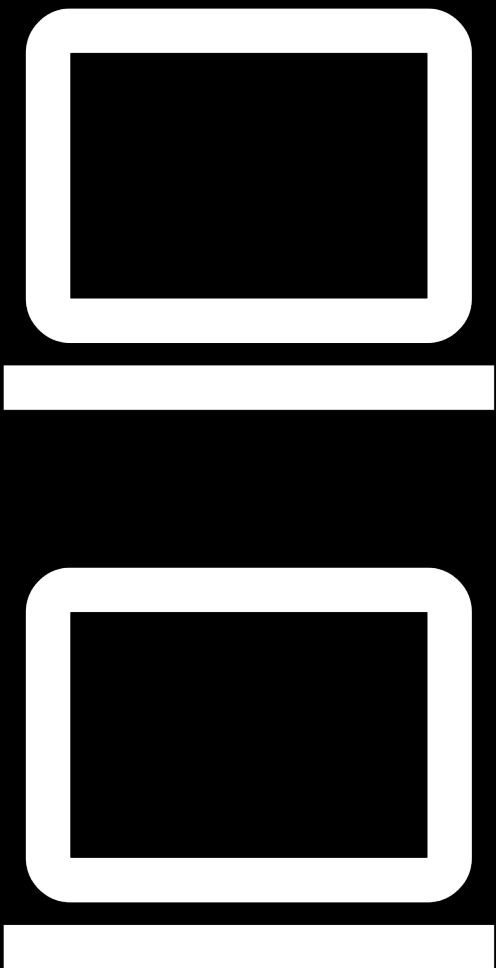


aiofiles	23.2.1	...
aiohttp	3.8.4	
aiosignal	1.3.1	
alabaster	0.7.12	
altair	5.1.2	
altgraph	0.17.4	
matplotlib	3.5.1	
matplotlib-inline	0.1.2	
numpy	1.22.4	
numpydoc	1.2	
pandas	1.4.2	
pygame	2.1.2	
pyqt-tools	1.0.0	
PyQt5	5.15.10	
PyQt5-Qt5	5.15.12	
PyQt5-sip	12.13.0	



프로그램 A
프로그램 B
프로그램 C
프로그램 D
프로그램 E

...

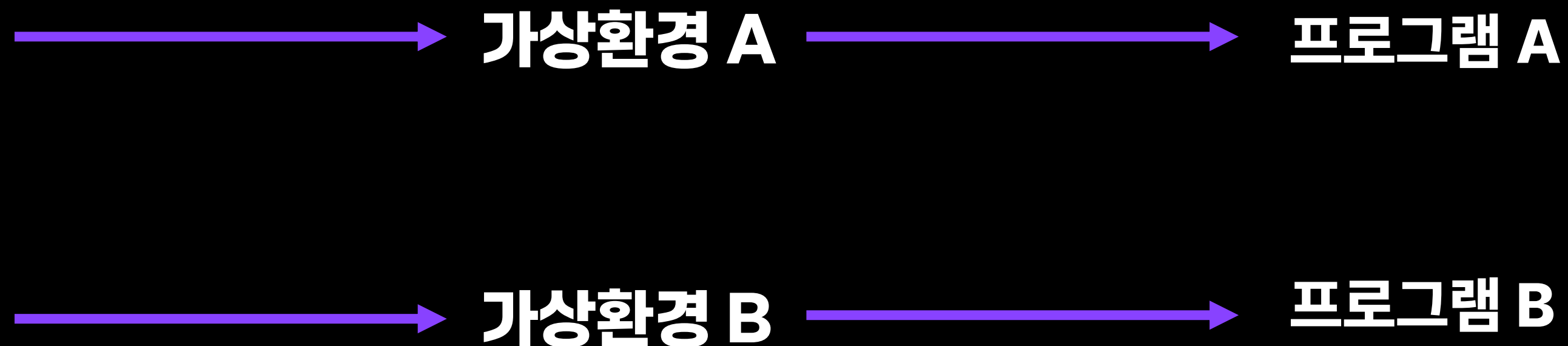
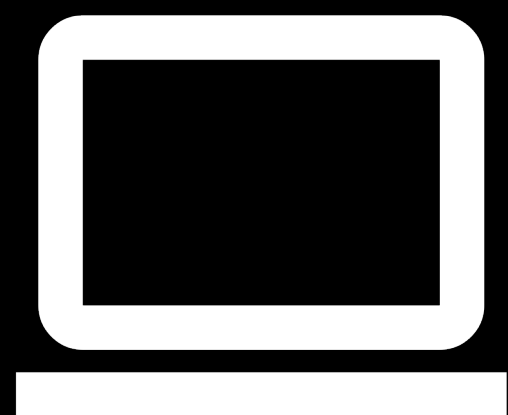


프로그램 A

프로그램 B



프로젝트 별로 격리된 환경을 제공하고,
각각의 환경에 필요한 라이브러리와 패키지들을 관리하기 위한 도구



3. 가상환경 활용하기



venv

vs

conda

- 파이썬 3.3 이상 버전의 표준 라이브러리
- 가상환경 생성, 활성화 등의 기능 지원
- 간단한 프로젝트나 파이썬의 표준 기능만을 이용하고자 할 때 주로 사용

- Anaconda의 일부로 제공되는 시스템
- 가상환경 생성, 활성화 등의 기능 지원
- 데이터 과학, 머신 러닝 프로젝트 등에서 특히 인기가 있음

데이터 과학 및 기계 학습을 위한 오픈 소스 배포판



- **Conda:** Anaconda의 패키지 및 환경 관리 도구
- **Jupyter Notebook:** 대화형 웹 기반 노트북 환경
- **NumPy, pandas, Matplotlib:** 데이터 처리, 분석, 시각화를 위한 핵심 라이브러리들이 포함
- **scikit-learn, TensorFlow, PyTorch:** 머신 러닝 및 딥 러닝을 위한 프레임워크 / 라이브러리들도 포함



venv

vs

conda

- 파이썬 3.3 이상 버전의 표준 라이브러리
- 가상환경 생성, 활성화 등의 기능 지원
- 간단한 프로젝트나 파이썬의 표준 기능만을 이용하고자 할 때 주로 사용

- Anaconda의 일부로 제공되는 시스템
- 가상환경 생성, 활성화 등의 기능 지원
- 데이터 과학, 머신 러닝 프로젝트 등에서 특히 인기가 있음



뒤에 오는 모듈을 실행하라는 의미

```
python -m venv <환경명>
```

설치된 파이썬 버전으로 생성됨

- 설정한 환경명으로 가상환경이 생성됨
- <환경명> 디렉토리를 생성하고, 그 안에 파이썬 인터프리터와 라이브러리를 포함한 가상환경을 설치



- 파이썬 코드를 실행하는 프로그램
- 사용자가 작성한 파이썬 코드 파일(.py) 또는 대화형 셸에서 입력한 명령어를 읽고, 이를 기계어로 변환하여 컴퓨터가 이해하고 실행하도록 함
- 코드를 한 줄씩 순차적으로 읽고 실행

대화형 모드

```
$ python3
Python 3.9.12 (main, Jun 1 2022, 06:34:44)
[Clang 12.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+3
5
>>> x = 10
>>> y = 20
>>> x + y
30
>>>
```

스크립트 모드

 test.py

```
$ python3 test.py
```



Windows

```
<환경명>\Scripts\activate
```

MacOS / Linux

```
source <환경명>/bin/activate
```

- 생성된 가상환경을 활성화함, OS별로 명령어가 상이
- 가상환경이 활성화되면, 터미널에 (<환경명>)이 표시되어
현재 활성화된 가상환경을 확인할 수 있음
- 활성화된 이후 라이브러리 설치 시 해당 가상환경의 site-packages 디렉토리에 설치됨



deactivate

활성화된 가상환경을 비활성화

Windows

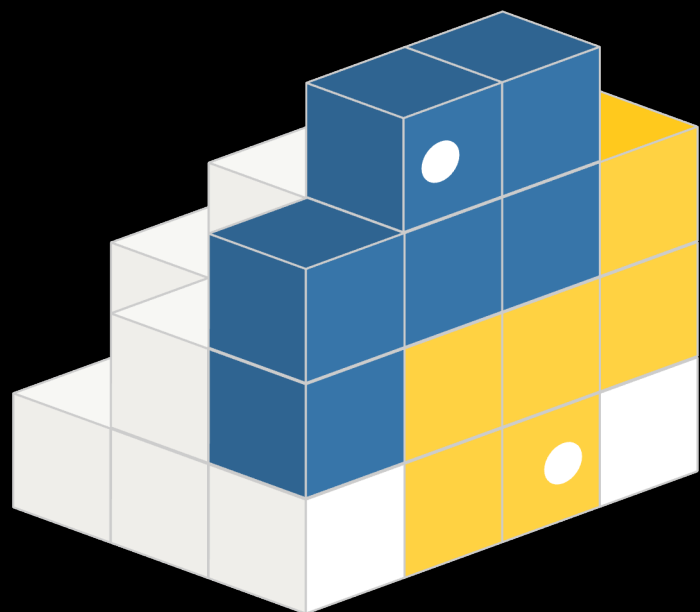
```
rmmdir /s <환경명>
```

MacOS / Linux

```
rm -rf <환경명>
```

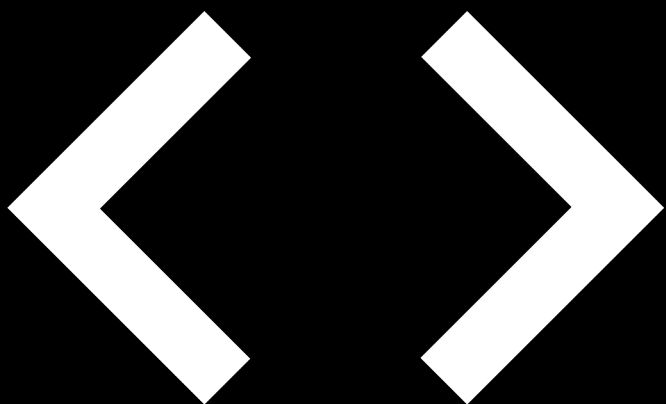
해당 디렉토리 즉, 가상환경을 삭제하는 방법

4. 라이브러리 설치 - PyPI

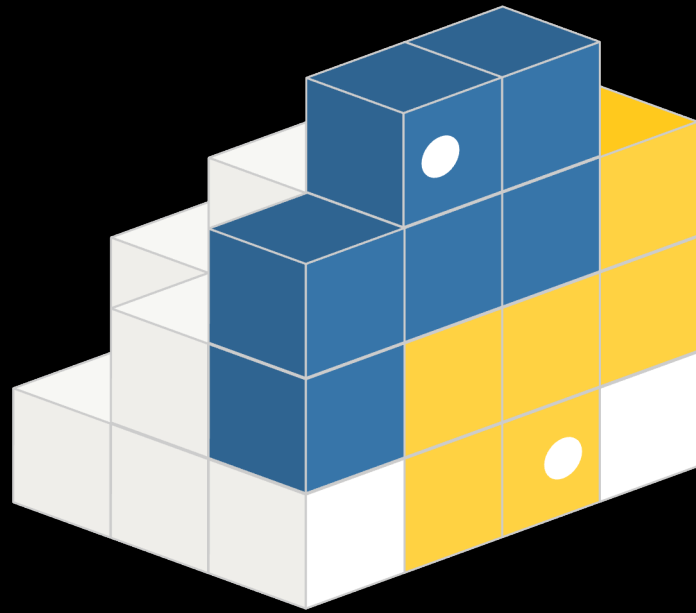


PyPI

VS



파일



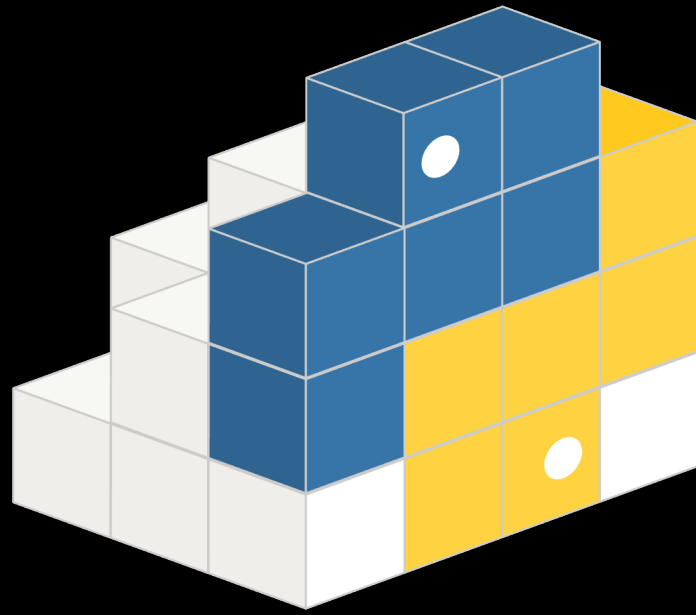
PyPI

= 파이썬 패키지를 호스팅하고 배포하는 공식 저장소

그렇다면, PyPI에 공유된 패키지는 어떻게 설치할 수 있을까?



정답은 pip!



pip

- PyPI에 공유된 패키지들이 저장된 저장소에서 패키지를 검색하고 설치
- pip를 활용하여 특정 패키지 설치, 업그레이드, 제거, 패키지 목록 관리 등이 가능



설치하기	pip install <패키지명>
특정 버전의 패키지 설치하기	pip install <패키지명>==버전
설치된 전체 패키지 & 버전 확인	pip list
설치된 특정 패키지 & 버전 확인	pip list grep <패지지명>
버전 업그레이드하기	pip install --upgrade <패키지명>
설치된 패키지 제거하기	pip uninstall <패키지명>



설치 과정 살펴보기

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

4. 라이브러리 설치 -
PyPI

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + v [ ] [ ] ... ^ X

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

o elicser@4b8b61724834:~$ [ ]
```

명시하지 않은 패키지들이 같이 설치되는 이유 = pip의 의존성 관리 기능



설치 과정 살펴보기

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

4. 라이브러리 설치 -
PyPI

```
Collecting charset-normalizer<4,>=2
  Downloading charset_normalizer-3.3.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
  (142 kB)
100% |#####| 142.1/142.1 kB 6.7 MB/s eta 0:00:00
Collecting certifi-2017.4.17
```

명시하지 않은 패키지들이 같이 설치되는 이유 = pip의 의존성 관리 기능



만약, 설치해야 할 패키지들이 많을 경우?

장보기 목록

우유 1.8L 2개
유정란 30구 3개
고기 만두 100g 5개
...

requirements.txt

```
aiofiles==23.2.1
aiohttp==3.8.4
aiosignal==1.3.1
alabaster==0.7.12
altair==5.1.2
altgraph==0.17.4
amqp==5.1.1
anaconda-client==1.9.0
anaconda-project==0.10.2
annotated-types== 0.6.0
anyio==3.7.1
appdirs==1.4.4
appnope==0.1.2
appscript==1.1.2
Automat==20.2.0
Babel==2.9.1
backcall==0.2.0
backoff==2.2.1
backports.functools-lru-cache==1.6.4
backports.tempfile==1.0
backports.weakref==1.0.post1
bcrypt==3.2.0
beautifulsoup4==4.11.1
billiard==3.6.4.0
```



- 설치해야 할 패키지들과 버전 목록을 명시한 파일
- requirements.txt는 암묵적으로 정해진 파일 이름

requirements.txt

```
aiofiles==23.2.1
aiohttp==3.8.4
aiosignal==1.3.1
alabaster==0.7.12
altair==5.1.2
altgraph==0.17.4
amqp==5.1.1
anaconda-client==1.9.0
anaconda-project==0.10.2
annotated-types== 0.6.0
anyio==3.7.1
appdirs==1.4.4
appnope==0.1.2
appscript==1.1.2
Automat==20.2.0
Babel==2.9.1
backcall==0.2.0
backoff==2.2.1
backports.functools-lru-cache==1.6.4
backports.tempfile==1.0
backports.weakref==1.0.post1
bcrypt==3.2.0
beautifulsoup4==4.11.1
billiard==3.6.4.0
```

pip install -r requirements.txt

-r : 해당 옵션 뒤에 오는 파일 경로를 읽어
해당 파일 내에 명시된 패키지를 설치



내 환경에서 설치된 패키지 목록과 버전을 한 번에 txt 파일로 내보낼 수 있음

requirements.txt

```
aiofiles==23.2.1
aiohttp==3.8.4
aiosignal==1.3.1
alabaster==0.7.12
altair==5.1.2
altgraph==0.17.4
amqp==5.1.1
anaconda-client==1.9.0
anaconda-project==0.10.2
annotated-types== 0.6.0
anyio==3.7.1
appdirs==1.4.4
appnope==0.1.2
appscript==1.1.2
Automat==20.2.0
Babel==2.9.1
backcall==0.2.0
backoff==2.2.1
backports.functools-lru-cache==1.6.4
backports.tempfile==1.0
backports.weakref==1.0.post1
bcrypt==3.2.0
beautifulsoup4==4.11.1
billiard==3.6.4.0
```

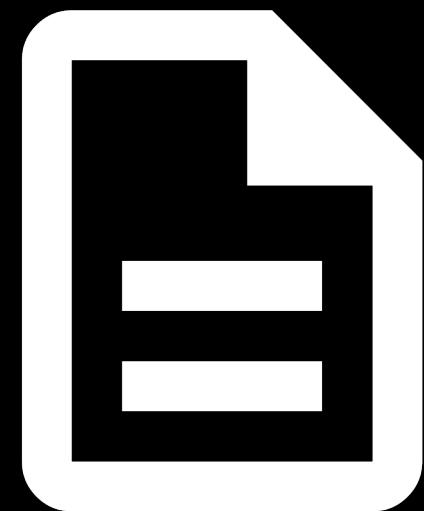
pip freeze > requirements.txt

현재 환경에 설치된 모든 패키지와 버전을 담은
requirements 라는 이름의 txt 파일 생성

5. 라이브러리 설치 - 파일



- PyPI에 등록되지 않았거나, 최신 버전을 사용하고 싶을 때
- .whl, .tar.gz 파일을 통해 설치



Library



- wheel 파일은 Python 패키지의 빌드된 배포판을 담고 있는 포맷
- pip가 설치 과정에서 소스 코드를 컴파일할 필요 없이 패키지를 직접 설치할 수 있음

< wheel 파일을 이용한 라이브러리 설치 명령어 >

```
pip install wheel_filename.whl
```




설치 과정 다시 살펴보기

실전 파이썬 라이브러리
활용

02 라이브러리 설치하기

5. 라이브러리 설치 - 파
일

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + v [ ] [ ] ... ^ X

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

o elicet@4b8b61724834:~$ [ ]
```

whl(wheel) 파일로 설치됨을 확인



{distribution}-{version}(-{build tag})?-{python tag}-{abi tag}-{platform tag}.whl

- {distribution}: 패키지의 이름
- {version}: 패키지의 버전
- {build tag}: 빌드 번호(선택적)
- {python tag}: 패키지가 호환되는 파이썬 버전(예: py2, py3, cp35)
- {abi tag}: 패키지가 호환되는 파이썬 ABI(Application Binary Interface) 버전
- {platform tag}: 패키지가 호환되는 플랫폼(예: any, win_amd64, manylinux1_x86_64)

requests-2.31.0-py3-none-any.whl

= requests 패키지 / 2.31.0 버전 / python3과 호환 / 모든 플랫폼에서 호환



Download files

Download the file for your platform. If you're not sure which to choose, learn more about [installing packages](#).

Source Distribution

 [pandas-2.2.0.tar.gz](#) (4.4 MB [view hashes](#))
Uploaded Jan 20, 2024 source

Built Distributions

 [pandas-2.2.0-cp312-cp312-win_amd64.whl](#) (11.5 MB [view hashes](#))
Uploaded Jan 20, 2024 cp312

 [pandas-2.2.0-cp312-cp312-musllinux_1_1_x86_64.whl](#) (13.4 MB [view hashes](#))
Uploaded Jan 20, 2024 cp312

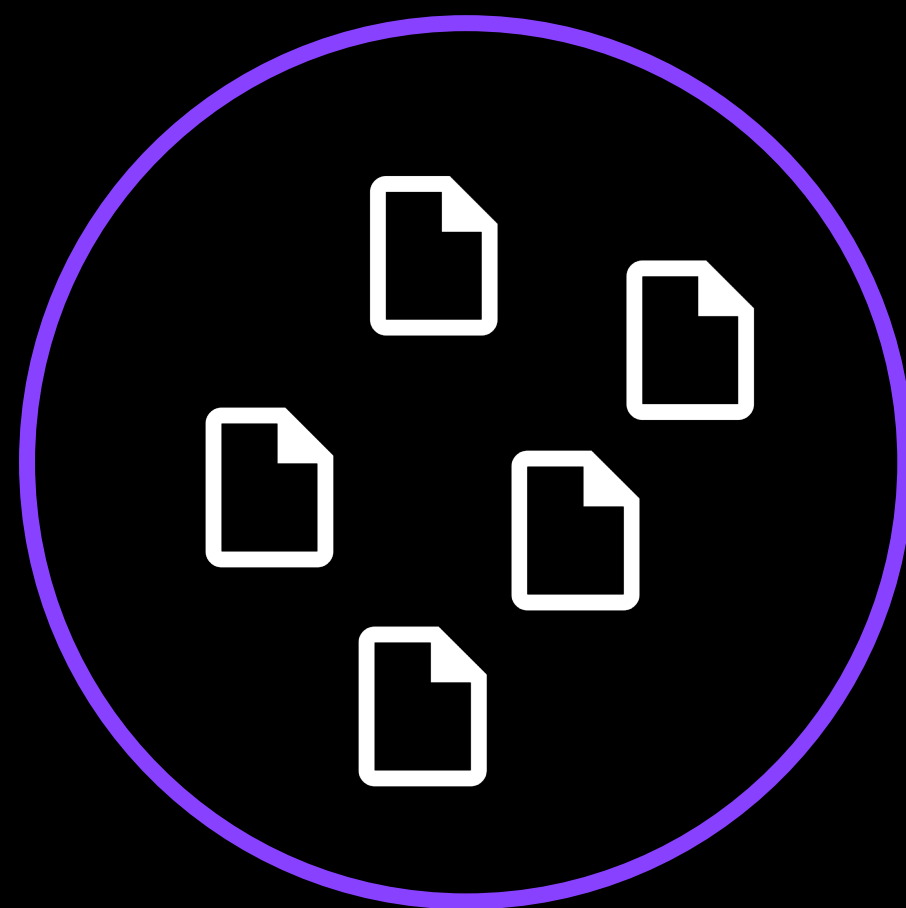
 [pandas-2.2.0-cp312-cp312-musllinux_1_1_aarch64.whl](#) (15.9 MB [view hashes](#))
Uploaded Jan 20, 2024 cp312

.tar.gz?

wheel



- Python 패키지의 소스 코드 배포판을 담고 있는 압축 파일 포맷
- 여러 파일과 디렉토리를 하나의 파일(.tar 파일, 또는 tarball이라고 함)로 묶고(tar), 그 다음 GNU zip(gzip)을 사용해 이 파일을 압축하여 .tar.gz 또는 .tgz 확장자를 생성



program.tar



gz 로 압축

program.tar.gz



- 패키지의 원본 코드를 포함하고 있으며, `setup.py` / `pyproject.toml`을 포함
- `tar -zxvf` 명령어로 압축을 해제한 이후 설치 가능
- 다양한 환경에서 빌드하여 사용할 수 있으나, 사용자가 적절한 컴파일러와 라이브러리를 설치해야 하며, 때로는 빌드 과정이 복잡하고 시간이 많이 소요

< 소스코드 파일을 이용한 패키지 설치 명령어 >

```
pip install .
```

- 패키지를 설치하고 빌드하기 위한 스크립트 파일
- 패키지의 메타데이터(이름, 버전, 저작권 정보) 의존성 라이브러리 등을 정의
- 패키지를 설치할 때 필요한 설정을 포함

`from setuptools import setup, find_packages` → **setuptools 라이브러리 사용**

```
setup(  
    name='mypackage',  
    version='1.0',  
    packages=find_packages(),  
    author='Author Name',  
    author_email='author@example.com',  
    description='Description of my package',  
    install_requires=[  
        'dependency1',  
        'dependency2',  
    ],  
)
```



- setup.py 파일은 파이썬의 오래된 방식, 최근엔 pyproject.toml 파일을 사용
- 빌드 시스템을 명시적으로 정의하고, 패키지 메타데이터 관리 기능 제공

```
[build-system]
requires = ["setuptools>=42", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "example-package"
version = "0.1.0"
description = "An example package"
authors = [{ name = "Author Name", email = "author@example.com" }]
license = { file="LICENSE" }
dependencies = [
    "requests>=2.24",
    "importlib-metadata; python_version<'3.8'"
]
```



	.tar.gz 파일	wheel 파일
형태	소스 코드 배포판 .tar 파일은 여러 파일과 디렉토리를 하나의 파일로 묶고, gzip을 사용해 압축한 것	바이너리 배포판 빌드된 패키지의 배포 포맷
사용 사례	사용자가 소스 코드를 직접 수정하거나 검토하고 싶을 때 사용	사용자는 컴파일 과정 없이 패키지를 바로 설치할 수 있음
장점	유연성이 높으며, 모든 시스템에서 빌드할 수 있음	설치가 빠르고 간단함
단점	사용자가 적절한 컴파일러와 라이브러리를 설치해야 하며, 설치 과정이 복잡하고 시간이 많이 소요될 수 있음	특정 플랫폼이나 파이썬 버전에 종속적일 수 있음



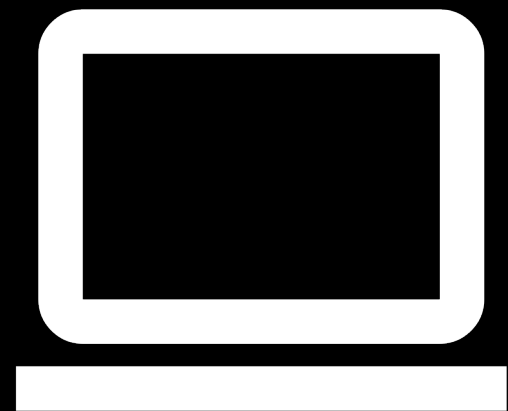
Github 에서 가져온 소스코드로 설치가 가능

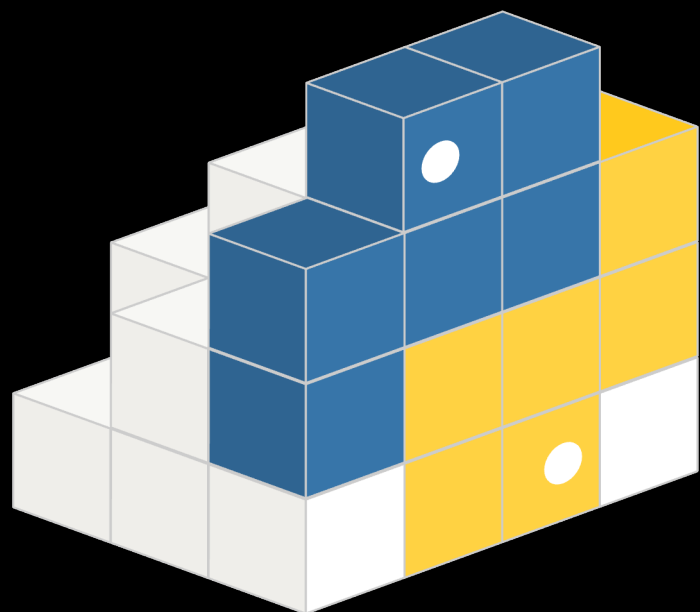
Git 저장소로부터 소스 코드 가져오기	<code>git clone https://github.com/username/library_name.git</code>
wheel 파일 설치	<code>pip install wheel_filename.whl</code>
.tar.gz 파일 압축 해제	<code>tar -zxvf 파일명.tar.gz</code>
소스코드로 설치	<code>pip install .</code>

6. 2장 정리 및 3장 예고



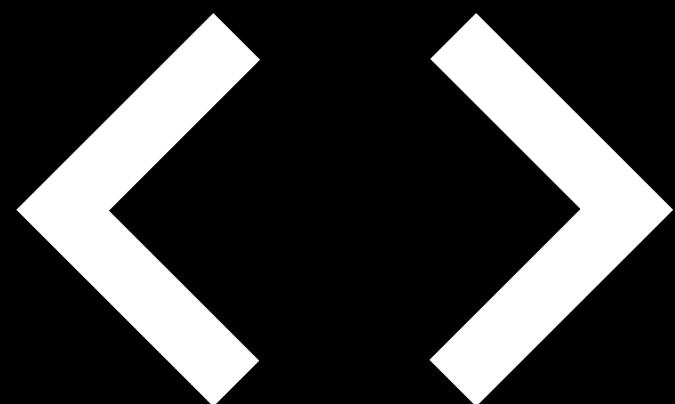
프로젝트 별로 격리된 환경을 제공하고,
각각의 환경에 필요한 라이브러리와 패키지들을 관리하기 위한 도구





PyPI

VS



파일

